

Assignment2_20241452 보고서

: Predict the ratings of movies in test data

- 소프트웨어학부 20241452 임지수 -

목차

1. 핵심 알고리즘 요약
2. 코드 함수 설명
3. 실행 결과 화면
4. 보완 및 테스트 피드백
5. 결론 및 깨달은 점

1. 핵심 알고리즘 요약

본 프로그램은 추천 시스템에서 널리 사용되는 협업 필터링 기술 중 하나인 편향을 고려한 행렬 분해(Biased Matrix Factorization) 알고리즘을 기반으로 합니다. 외부 딥러닝 프레임워크에 의존하지 않고 NumPy만을 활용하여 수학적 최적화 과정을 바닥부터 직접 구현하였으며, 알고리즘의 핵심 메커니즘은 다음과 같습니다.

가. 평점 예측 모델 수식 (Prediction Model)

: 단순한 행렬 분해 모델은 유저의 성향(항상 평점을 높게 주거나 낮게 주는 경향)이나 아이템의 고유 특성(작품성이 뛰어나 대체로 평점이 높은 경향)을 반영하지 못합니다. 이를 보완하기 위해 전체 평균, 유저 편향, 아이템 편향 항을 결합한 개별 평점 예측 수식을 정의하여 사용합니다.

$$\hat{r}_{ui} = \mu + b_u + b_i + P_u Q_i^T$$

- μ (Global Bias): 학습 데이터셋 전체 평점의 산술 평균값. 코드의 `fit()` 함수 시작 시 가장 먼저 계산되어 고정.
- b_u (User Bias): 특정 유저 u 가 가지는 평점 부여 성향 편향.
- b_i (Item Bias): 특정 아이템 i 가 가지는 고유한 평점 분포 편향.
- $P_u \in \mathbb{R}^k$: 유저 u 의 특징을 나타내는 k 차원의 잠재 요인 벡터.
- $Q_i \in \mathbb{R}^k$: 아이템 i 의 특징을 나타내는 k 차원의 잠재 요인 벡터.

나. 목적 함수 및 최적화 알고리즘 (Optimization via SGD)

: 모델의 학습 목적은 실제 평점과 모델의 예측 평점 사이의 오차를 최소화하는 것입니다. 과적합을 방지하기 위해 L2 Regularization이 포함된 손실 함수를 정의하고, 확률적 경사 하강법(SGD)을 통해 파라미터를 실시간으로 업데이트합니다.

1) 목적 함수 (Objective Function)

$$Loss = \sum_{(u,i) \in R} (r_{ui} - \hat{r}_{ui})^2 + \lambda(\|P_u\|^2 + \|Q_i\|^2 + b_u^2 + b_i^2)$$

2) 확률적 경사 하강법(SGD) 적용 및 가중치 업데이트 규칙

: 매 Epoch마다 학습 데이터를 무작위로 섞은 후, 데이터 한 개씩 순차적으로 예측 오차를 계산합니다. 목적 함수를 각 파라미터로 편미분하여 도출된 그라디언트의 반대 방향으로 학습률(lr)만큼 이동하며 가중치를 갱신합니다. 상수 2는 학습률에 흡수시켜 축약합니다.

* 코드 구현 상의 디테일 (Deep Dive)

잠재 요인 행렬 P_u 를 업데이트할 때, 가중치 변화가 동시에 일어나므로 연산의 왜곡을 막기 위해 `temp_p = self.P[u, :].copy()`를 활용하여 업데이트 이전의 원본 값을 백업한 뒤 Q_i 변수 갱신 연산에 적용하도록 구현하였습니다.

다. 예외 처리 및 후처리 메커니즘 (Robustness)

1) Cold-Start Handling (신규 데이터 대응 예외 처리)

: 추천 시스템이 직면하는 고질적인 문제인 '학습 데이터에 존재하지 않는 새로운 유저나 아이템이 테스트 세트에 등장하는 현상'을 방어하기 위해 `predict()` 함수 내부에 4단계 분기 로직을 설계했습니다.

- 유저와 아이템 모두 기면 (Known): 기본 예측 수식을 적용.
- 새로운 아이템 등장 (유저만 인지): 아이템 편향과 잠재 요인을 제외한 유저의 주관적 성향만 반영.
- 새로운 유저 등장 (아이템만 인지): 유저 편향과 잠재 요인을 제외한 아이템의 대중적 평점 성향만 반영.
- 둘 다 처음 보는 데이터 (Unknown): 데이터셋 전체의 평균 평점을 반환하여 시스템 에러를 방지.

2) 평점 범위 제한 (Prediction Clipping)

: 행렬 연산 및 편향 합산 과정에서 간혹 모델의 예측값이 평점의 유효 범위(1.0점 ~ 5.0점)를 벗어나는 오버슈팅 현상이 발생할 수 있습니다. 이를 제어하기 위해 예측 최종 단계에서 `np.clip` 연산을 적용하여 스코어를 $1.0 \leq \hat{r}_{ui} \leq 5.0$ 범위 내로 강제 구속함으로써 평가 메트릭(RMSE)의 안정성을 극대화했습니다.

2. 코드 함수 설명

본 프로그램은 모듈화된 전역 함수와 MatrixFactorization 클래스 내의 핵심 메서드들로 구성되어 있습니다. 각 함수의 역할과 입출력 정의, 핵심 구현 디테일은 다음과 같습니다.

가. 데이터 로딩 및 전처리 전역 함수

1) load_data(filename)

- **역할:** 지정된 텍스트 파일로부터 MovieLens 데이터를 읽어와 파싱하고, 전체 유저 및 아이템의 최대 식별자(ID) 크기를 측정합니다.
- **입력 매개변수:** filename (데이터셋 파일 경로, str)
- **출력 반환값:** data (학습 데이터 리스트, list), max_u_id (최대 유저 ID, int), max_i_id (최대 아이템 ID, int)
- **주요 로직:** 파일을 한 줄씩 읽어 탭(wt) 문자를 기준으로 유저, 아이템, 평점을 분리한 뒤 정수 및 실수형으로 변환하여 튜플 형태로 리스트에 저장합니다. 행렬의 메모리 할당 크기를 결정하기 위해 데이터셋 내의 최대 ID 값을 실시간으로 추적하여 함께 반환합니다.

나. MatrixFactorization 클래스 내부 메서드

2) __init__(self, n_users, n_items, k=40, lr=0.014, reg=0.10, epochs=30, seed=3, min_rating=1.0, max_rating=5.0)

- **역할:** 모델 객체를 생성하고, 하이퍼파라미터를 등록하며, 학습에 필요한 편향 배열과 잠재 요인 행렬들을 초기화합니다.
- **입력 매개변수:** 유저/아이템 수 및 알고리즘 제어 파라미터(차원수, 학습률, 정규화 계수, 에포크, 시드값, 평점 상하한선 등)
- **주요 로직:**
 - 성능의 재현성(Reproducibility)을 보장하기 위해 최신 NumPy 표준인 np.random.default_rng(seed) 객체를 생성하여 내부 난수 발생기(self.rng)로 설정합니다.
 - 인덱스 오버플로우를 방지하기 위해 유저 및 아이템 크기에 + 1 을 더한 크기로 가중치 공간을 확보합니다.
 - 잠재 요인 행렬 P 와 Q 는 평균 0.0, 표준편차 0.1 의 가우시안 정규분포 난수로 초기화하며, 유저 편향(b_u)과 아이템 편향(b_i), 전체 평균 편향(b)은 모두 0 의 초기값으로 설정합니다.

3) fit(self, training_data)

- **역할:** 확률적 경사 하강법(SGD)을 실행하여 모델의 모든 파라미터(전체 평균, 편향, 잠재 요인)를 학습 데이터에 맞게 최적화합니다.
- **입력 매개변수:** training_data (유저, 아이템, 평점 튜플로 구성된 학습용 데이터 세트)
- **주요 로직:**
 - 가장 먼저 데이터셋 내 모든 실제 평점의 산술 평균을 구하여 전체 평균 편향(self.b)을 확정합니다.
 - 지정된 epochs 횟수만큼 대루프를 수행하며, 매 에포크 시작 시 self.rng.shuffle()을 호출하여 학습 데이터의 순서를 무작위로 섞음으로써 SGD 알고리즘이 국소 최적해에 빠지는 것을 방지합니다.
 - 무작위 정렬된 데이터를 순회하며 실제 평점과 예측값의 오차 e 를 구하고, 목적 함수의 그래디언트 방향에 따라 유저/아이템 편향 및 잠재 요인 벡터를 동시 업데이트합니다. 이때 P 의 가중치 변화가 Q 의 연산에 교란을 주는 것을 막기 위해 .copy() 메서드로 업데이트 전의 원본 P 벡터(temp_p)를 안전하게 백업하여 연산에 활용합니다.

4) _clip(self, score)

- **역할:** 계산된 예측 평점이 유효한 평점 범위를 벗어날 경우 이를 상하한선 내로 강제 구속하는 내부 유틸리티 메서드입니다.
- **입력 매개변수:** score (모델 수식에 의해 연산된 직후의 예측 평점, float)
- **출력 반환값:** 제한 범위 내로 보정된 최종 평점 (float)
- **주요 로직:** np.clip 함수를 활용하여 입력값이 시스템의 최소 평점(self.min_rating=1.0)보다 작으면 1.0을, 최대 평점(self.max_rating=5.0)보다 크면 5.0을 반환하도록 제어하여 예측 오차의 왜곡을 방지합니다.

5) predict(self, u, i)

- **역할:** 입력된 특정 유저 ID와 아이템 ID에 대한 최종 평점을 예측하며, 인덱스 범위 조회를 통해 Cold-Start 문제를 방어합니다.
- **입력 매개변수:** u (유저 인덱스, int), i (아이템 인덱스, int)
- **출력 반환값:** Cold-Start 예외 처리 및 후처리가 완료된 예측 평점 (float)
- **주요 로직:** 입력받은 인덱스가 학습 단계에서 인지한 범위 내에 존재하는지 조건문(user_known, item_known)을 통해 판단합니다. 조건 충족 여부에 따라 총 4개의 분기문(전체 수식 적용, 유저 성향만 반영, 아이템 성향만 반영, 전체 평균 적용)을 유연하게 거치며, 최종 연산값은 _clip() 메서드를 거쳐 유효 범위 내의 값으로 변환되어 리턴됩니다.

다. 시스템 제어 및 실행 전역 함수

6) `parse_args()`

- **역할:** 터미널 환경(CLI)에서 사용자가 직접 모델 파라미터 및 실행 파일을 주입할 수 있도록 명령행 인자를 파싱합니다.
- **출력 반환값:** 파싱된 인자 정보들을 담은 객체 (`argparse.Namespace`)
- **주요 로직:** `argparse` 라이브러리를 빌드하여 필수 인자인 학습 파일(`base_file`)과 테스트 파일(`test_file`)을 정의합니다. 또한 잠재 요인 차원수(`--k`), 학습률(`--lr`), 정규화 계수(`--reg`), 에포크 수(`--epochs`), 시드값(`--seed`)을 선택적 플래그 인자로 선언하고, 미입력 시 최적의 성능을 냈던 하이퍼파라미터 디폴트 값이 자동으로 할당되도록 명시했습니다.

7) `main()`

- **역할:** 프로그램의 메인 엔트리 포인트(Entry Point)로서, 데이터 로드부터 모델 생성, 학습, 예측 및 파일 출력까지의 전체 추천 시스템 파이프라인을 총괄 제어합니다.
- **주요 로직:**
 - `parse_args()`를 통해 사용자가 입력한 실행 옵션을 파싱합니다.
 - 학습 데이터와 테스트 데이터를 각각 `load_data()`로 읽어들이고, 두 데이터셋을 아우르는 글로벌 최대 유저/아이템 인덱스를 계산하여 행렬의 범위를 안전하게 확장합니다.
 - `time.time()`을 이용해 모델의 순수 학습 소요 시간을 측정하여 화면에 출력합니다.
 - 학습이 완료되면 테스트 세트 파일을 다시 한 줄씩 읽으며 `model.predict()`를 통해 평점을 예측하고, 그 결과를 탭 문자로 구분하여 {입력파일명}_prediction.txt 규격의 최종 결과 파일로 출력 및 저장합니다.

3. 실행 결과 화면

```
PS C:\Users\지수\Desktop\Assignment2_20241452> python test_rs.py u
1.test u1.base_prediction.txt
Loading Ground Truth: u1.test
Loading Predictions: u1.base_prediction.txt
-----
Total evaluated pairs: 20000
🔥 FINAL RMSE: 0.923729
```

```
PS C:\Users\지수\Desktop\Assignment2_20241452> python test_rs.py u
2.test u2.base_prediction.txt
Loading Ground Truth: u2.test
Loading Predictions: u2.base_prediction.txt
-----
Total evaluated pairs: 20000
🔥 FINAL RMSE: 0.914145
```

```
PS C:\Users\지수\Desktop\Assignment2_20241452> python test_rs.py u
3.test u3.base_prediction.txt
Loading Ground Truth: u3.test
Loading Predictions: u3.base_prediction.txt
-----
Total evaluated pairs: 20000
🔥 FINAL RMSE: 0.905936
```

```
PS C:\Users\지수\Desktop\Assignment2_20241452> python test_rs.py u
4.test u4.base_prediction.txt
Loading Ground Truth: u4.test
Loading Predictions: u4.base_prediction.txt
-----
Total evaluated pairs: 20000
🔥 FINAL RMSE: 0.908046
```

```
PS C:\Users\지수\Desktop\Assignment2_20241452> python test_rs.py u
5.test u5.base_prediction.txt
Loading Ground Truth: u5.test
Loading Predictions: u5.base_prediction.txt
-----
Total evaluated pairs: 20000
🔥 FINAL RMSE: 0.911844
```

- u1~u5 평균 RMSE : **0.912738**

4. 보완 및 테스트 피드백

본 프로젝트를 통해 외부 프레임워크 없이 Biased Matrix Factorization 알고리즘을 성공적으로 구현하고 목표 Baseline을 달성하였으나, 실제 상용화 시스템 및 대규모 데이터셋 적용 관점에서 다음과 같은 한계점과 향후 보완 방향을 도출하였습니다.

가. 연산 효율성 및 확장성 개선

- **한계점:** 현재 구현된 SGD 학습 방식은 에포크마다 전체 데이터셋을 파이썬 내장 for 루프를 순회하며 가중치를 갱신합니다. MovieLens 100K(10 만 개 데이터) 환경에서는 30 에포크 기준 수십 초 내에 학습이 완료되지만, 수백만에서 수천만 건 이상의 대규모 데이터셋 환경에서는 파이썬 오버헤드로 인해 연산 시간이 기하급수적으로 증가하는 치명적인 성능 병목이 발생할 수 있습니다.
- **보완 방향:** NumPy의 행렬 연산 연동을 극대화하여 미니배치 단위로 연산을 벡터화하거나, C 언어 수준의 연산 속도를 보장하는 Cython의 도입, 혹은 멀티코어 CPU/GPU 가속을 지원하는 병렬 처리 아키텍처로의 전환이 필요합니다.

나. 적응형 학습률(Adaptive Learning Rate) 최적화 알고리즘 도입

- **한계점:** 현재 모델은 고정된 학습률($lr=0.014$)을 사용하며, 과적합과 오차 발산을 오직 정규화 계수($reg=0.10$)의 규제 강도에만 의존하여 방어하고 있습니다. 이로 인해 최적의 성능을 내는 파라미터를 찾기 위해 수많은 수동 튜닝 과정이 강제되는 번거로움이 존재합니다.
- **보완 방향:** 학습 초반에는 빠른 수렴을 위해 보폭을 크게 가져가고, 최적점에 가까워질수록 보폭을 미세하게 조정하는 학습률 감쇠 로직을 추가할 수 있습니다. 더 나아가 각 가중치 파라미터의 기울기 변화량에 따라 학습률을 스스로 동적으로 조절하는 AdaGrad, RMSProp, 또는 Adam 최적화 알고리즘을 결합한다면, 수동 파라미터 튜닝 스트레스를 줄이고 더욱 안정적인 수렴 성능을 확보할 수 있을 것입니다.

다. Cold-Start 문제의 근본적인 해결을 위한 하이브리드(Hybrid) 아키텍처 확장

- **한계점:** 현재 `predict()` 함수는 훈련 데이터에 없는 새로운 유저나 아이템이 들어왔을 때, 시스템 에러를 방지하기 위해 전체 평균 평점이나 단편적인 편향값만을 반환하는 예외 처리 로직이 구현되어 있습니다. 이는 시스템의 안정성 측면에서는 좋은 방어기제이지만,

해당 유저에게 특화된 개인화 추천이라는 추천 시스템 본연의 가치를 제공하는 데에는 한계가 있습니다.

- **보완 방향:** 유저의 인구통계학적 정보(나이, 성별, 직업)나 아이템의 메타데이터(영화 장르, 감독, 감독, 개봉 연도) 등의 콘텐츠 정보를 추가적으로 수집하여, 가용 정보가 없을 때는 콘텐츠 기반 필터링을 수행하고 데이터가 쌓이면 행렬 분해를 수행하는 하이브리드 추천 시스템으로 아키텍처를 확장할 수 있습니다.

라. 하이퍼파라미터 검증 프로세스의 자동화 파이프라인 구축

- **한계점:** 본 실험에서는 k , lr , reg , $epochs$ 등의 파라미터를 터미널 플래그로 입력받아 수동으로 실행하며 검증 결과를 도출하였습니다. 교차 검증(5-Fold Cross Validation) 환경에서 모든 파라미터 조합의 전역 최적해를 탐색하기에는 시간적, 물리적 제약이 따랐습니다.
- **보완 방향:** 최적의 하이퍼파라미터 조합을 자동으로 전수 조사하는 Hyperparameter Auto-Tuning 스크립트를 상위에 구축하고, Random Search 기법이나 베이지안 최적화 라이브러리를 연동하여 최적의 성능을 내는 스위트스팟을 수학적·체계적으로 도출하는 파이프라인 개선이 요구됩니다.

5. 결론 및 깨달은 점

본 프로젝트는 알고리즘 라이브러리의 추상화된 함수 호출 방식에서 벗어나, 선형대수 수식과 최적화 이론을 바탕으로 Biased Matrix Factorization 추천 시스템을 직접 구현해보는 프로젝트였습니다. 과제를 수행하며 크게 깨달은 점은 다음과 같습니다.

- 수학적 이론과 소프트웨어 구현의 유기적 연결

: 강의실에서 텍스트로만 접했던 행렬 분해, 편미분, 확률적 경사 하강법(SGD) 등의 수학적 개념들이 실제 코드의 루프문과 가중치 업데이트 식으로 1:1 매칭되어 구동되는 과정을 직접 확인하였습니다. 이 과정에서 복잡한 인공지능 모델의 본질 역시 결국 수학적 최적화 알고리즘에 기반한다는 점을 이해하게 되었습니다.

- 아키텍처와 프레임워크 이해

: 그동안 당연하게 사용해왔던 거대한 AI 프레임워크들이 내부적으로 얼마나 고도화된 최적화 연산과 예외 처리를 수행하고 있었는지 그 이면의 아키텍처를 역으로 체감할 수 있었습니다. 라이브러리를 사용할 때도 그 내부 메커니즘을 명확히 인지하고 제어할 수 있는 엔지니어링 마인드셋을 확립하는 계기가 되었습니다.

- 하이퍼파라미터 튜닝을 통한 엔지니어링 끈기와 데이터 감각 체득

: 초기 모델 설계 시 오차가 발산하거나 수렴 속도가 지나치게 느려지는 현상을 겪으며, k , lr , reg 등의 하이퍼파라미터가 모델의 해상도와 정밀도에 미치는 영향을 독립적으로 분석하였습니다. 수많은 반복 실험과 튜닝 과정을 통해 목표 Baseline(RMSE 0.9130)을 돌파하여 최종 **0.912738**이라는 최적의 스윕스팟을 찾아내었으며, 시스템의 성능 향상은 단순한 코드 작성이 아닌 정교한 데이터 분석과 집념의 결과물임을 깨달았습니다.